

INSTITUTO FEDERAL DE EDUCAÇÃO CIÊNCIA E TECNOLOGIA DE MINAS
GERAIS - CAMPUS SÃO JOÃO EVANGELISTA
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

Bianca de Castro Aguiar Fontes
Wanessa Silva Ferreira

**RELATÓRIO FINAL DE DESENVOLVIMENTO WEB II: sistema de gerenciamento
para clínica veterinária**

São João Evangelista

2026

LISTA DE ILUSTRAÇÕES

Figura 1 - Página inicial.....	4
Figura 2 - Estrutura de pastas do projeto.....	5
Figura 3 - Comparação do estado do menu superior sendo reutilizado.....	6
Figura 4 - Tela de listagem de raças com tabela Bootstrap Table paginada.....	7
Figura 5 - Exemplo de formulário com validação exibindo mensagem de erro.....	8
Figura 6 - Página de login.....	9
Figura 7 - Tentativa de acesso a rota protegida sem login, sendo redirecionado.....	10
Figura 8 - Tela de listagem de espécies com as ações de editar e excluir.....	11
Figura 9 - Tela de edição de espécie.....	11
Figura 10 - Confirmação de exclusão de espécie.....	12
Figura 11 - Tela de listagem de consultas.....	13
Figura 12 - PDF gerado de uma ficha de consulta.....	14
Figura 13 - E-mail de redefinição de senha recebido pelo usuário.....	15
Figura 14 - Campo de raças sendo preenchido dinamicamente após selecionar a espécie.....	16
Figura 15 - Token CSRF gerado ao efetuar login.....	17
Figura 16 - Página 404 personalizada.....	18

SUMÁRIO

1 INTRODUÇÃO.....	2
2 DESENVOLVIMENTO.....	3
2.1 Tecnologias utilizadas no desenvolvimento (Requisito 1).....	3
2.2 Arquitetura em camadas (MVC e Service) (Requisito 2).....	3
2.3 Utilização de layouts (Requisito 3).....	4
2.4 Utilização de Framework Frontend (Requisito 4).....	4
2.5 Boas práticas de usabilidade (Requisito 5).....	5
2.6 Páginas públicas (Requisito 6).....	5
2.7 Área restrita (Requisito 7).....	5
2.8 Operações de CRUD (Requisito 8).....	6
2.9 Relacionamento entre tabelas (Requisito 9).....	6
2.10 Exportação em PDF (Requisito 10).....	7
2.11 Autenticação de usuário (Requisito 11).....	7
2.12 Requisições AJAX (Requisito 12).....	8
2.13 Filtro de proteção contra CSRF (Requisito 13).....	8
2.14 Proteção contra XSS.....	9
2.15 Página de error personalizada (Requisito 14).....	10
3 FUNCIONALIDADES NÃO CONCLUÍDAS, LIMITAÇÕES E DIFICULDADES....	11
3.1 Funcionalidades não concluídas ou com escopo reduzido.....	11
3.2 Limitações identificadas.....	12
3.3 Principais dificuldades enfrentadas.....	12
4 CONCLUSÃO.....	13

1 INTRODUÇÃO

O presente relatório apresenta o Trabalho Final da disciplina de Desenvolvimento Web II, cujo objetivo é colocar em prática o que foi desenvolvido e aprendido em sala de aula durante todo o semestre, utilizando o *framework* CodeIgniter 4. Para este trabalho foi escolhido o desenvolvimento de um sistema de gerenciamento para clínica veterinária, denominado Faro Animal, permitindo que médicos veterinários realizem o cadastro e o gerenciamento de informações relacionadas aos animais atendidos.

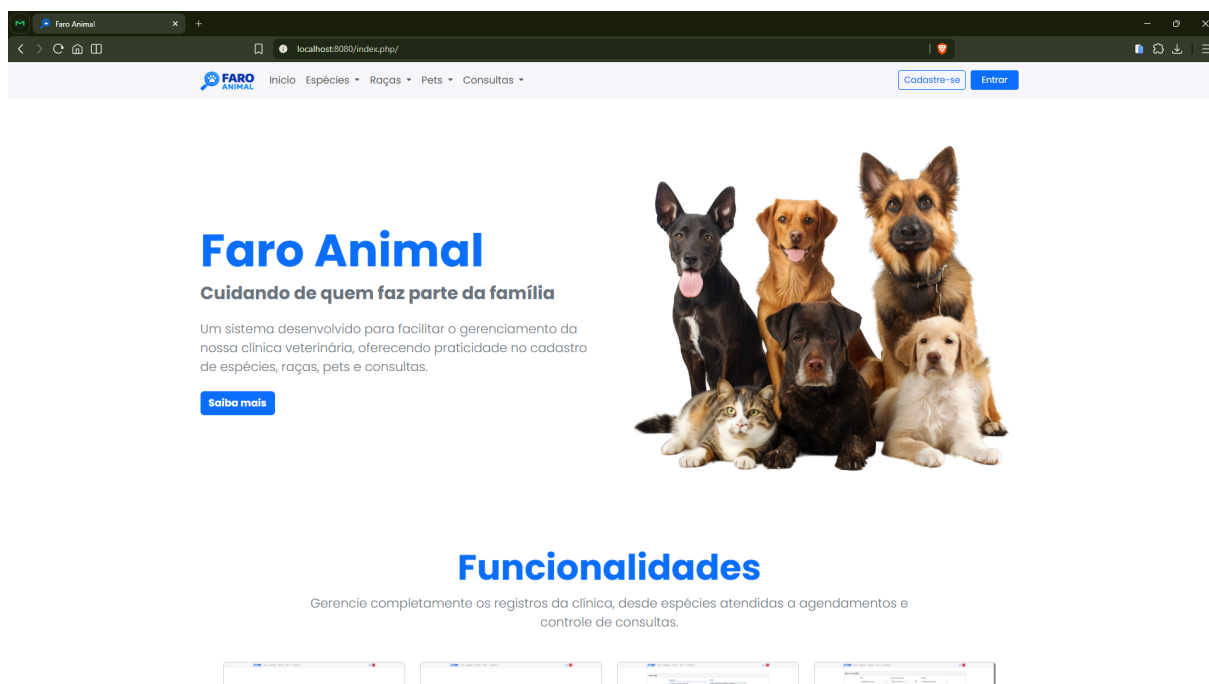
Durante o desenvolvimento foram aplicados os conhecimentos adquiridos ao longo da disciplina, envolvendo *frontend* e *backend*, arquitetura MVC (Model-View-Controller), integração com banco de dados, autenticação de usuários, implementação de *Services*, comunicação via AJAX, geração de relatórios de consultas em PDF, além de proteção contra CSRF, XSS e tratamento de erros. No decorrer deste relatório serão apresentados os requisitos definidos pelo roteiro proposto pelo professor, descrevendo como cada um foi implementado e como é o funcionamento do sistema. Ao final, são discutidas as limitações encontradas e as dificuldades enfrentadas pela equipe durante o desenvolvimento.

2 DESENVOLVIMENTO

2.1 Tecnologias utilizadas no desenvolvimento (Requisito 1)

Para atender ao requisito 1, o sistema foi desenvolvido utilizando HTML para a estruturação das páginas, CSS e *Bootstrap 5* para a estilização e organização da interface, *JavaScript* para a implementação de funcionalidades dinâmicas no lado do cliente e PHP para a programação do lado do servidor. Todo o projeto foi construído sobre o *framework* CodeIgniter 4, responsável por organizar a aplicação e facilitar o desenvolvimento em camadas, seguindo boas práticas de mercado (roteamento nomeado, filtros, serviços e *Query Builder* para acesso ao banco de dados).

Figura 1 - Página inicial



Fonte: Elaborado pelo autor, 2026

2.2 Arquitetura em camadas (MVC e Service) (Requisito 2)

A aplicação foi organizada utilizando o padrão de arquitetura MVC, como solicita o requisito 2, no qual as responsabilidades são separadas em acesso aos dados, regras de negócio e interface do usuário. Os *Models* (*AppointmentModel*, *PetModel*, *BreedModel*, *SpecieModel* e *UserModel*) são responsáveis pela comunicação com o banco de dados, definindo campos permitidos, *timestamps* automáticos e exclusão lógica (*soft delete*). Os *Controllers* recebem e tratam as requisições dos usuários, delegando toda a lógica de negócio às classes de *Service* correspondentes (*AppointmentService*, *PetService*, *BreedService*, *SpecieService*, *UserService* e *AuthService*), que foram registradas no arquivo *Services.php* e são acessadas por meio do *helper service()*. Já as *Views* são responsáveis exclusivamente pela exibição das informações na interface.

A utilização de *Services* permitiu concentrar as regras de negócio (validações, verificação de duplicidade, cruzamento entre tabelas, tratamento de exceções) fora dos *controllers*, o que torna o código mais organizado, reutilizável e de fácil manutenção. Cada método de *Service* retorna um *array* padronizado contendo as chaves *success*, *message*, *data*, *invalidArgs* e *errors*, o que uniformiza o tratamento de respostas em toda a aplicação.

Figura 2 - Estrutura de pastas do projeto

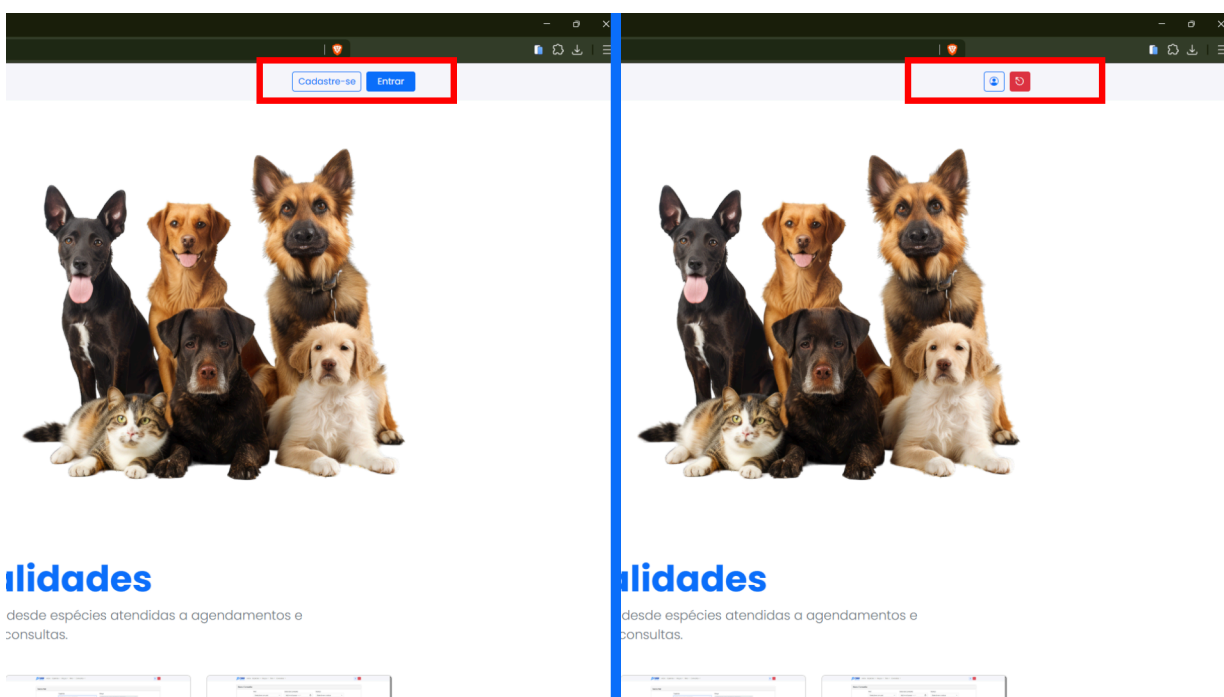


2.3 Utilização de *layouts* (Requisito 3)

No sistema foi utilizado o conceito de *layouts* do CodeIgniter 4 para padronizar todas as páginas da aplicação. O arquivo *layouts/main.php* centraliza a estrutura HTML principal (*head*, importação de estilos e *scripts*) e utiliza os métodos *extend()*, *section()* e *renderSection()* para que cada *view* defina apenas o seu conteúdo específico. Componentes visuais como o menu superior (*partials/navbar.php*) e o rodapé (*partials/footer.php*) são incluídos por meio de *\$this->include()*, sendo reutilizados em todas as telas. Essa abordagem evita a repetição desnecessária de código HTML e facilita a manutenção caso seja necessário alterar a identidade visual do sistema no futuro.

É interessante dizer que os botões presentes no menu superior são dinâmicos, ou seja, são alterados caso tenha um usuário logado ou não. Isso foi possível pela condição da variável *\$logged*, que retorna o resultado de *session()->has('user_id')*;

Figura 3 - Comparação do estado do menu superior sendo reutilizado

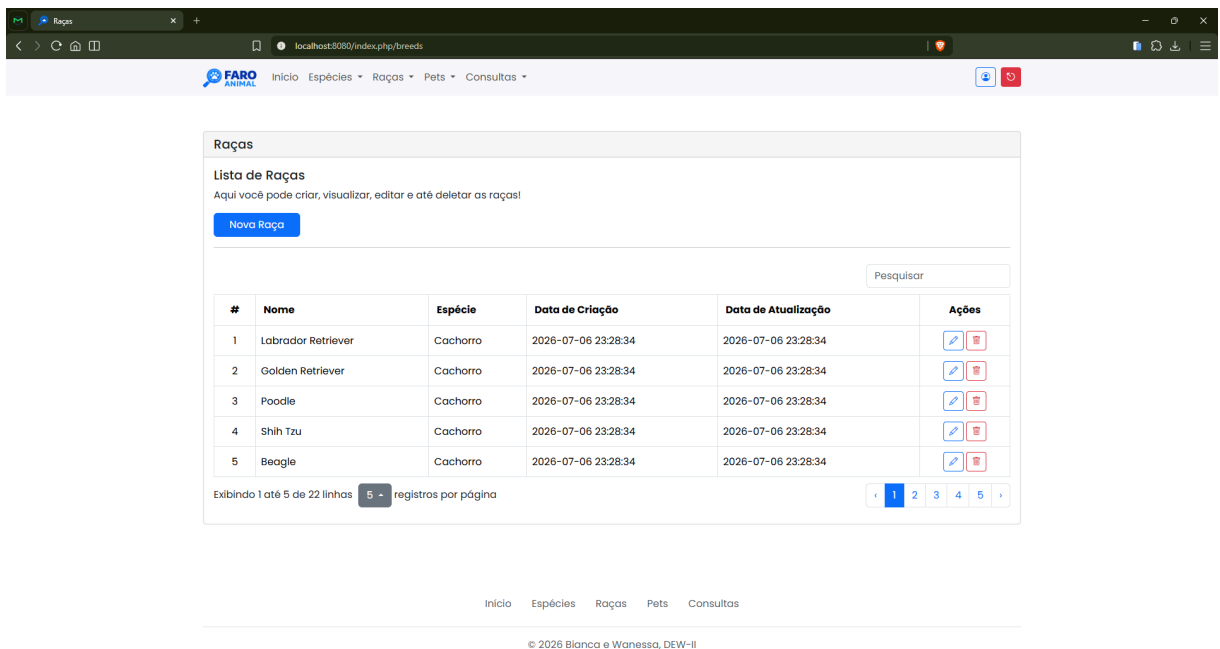


Fonte: Elaborado pelo autor, 2026

2.4 Utilização de *Framework Frontend* (Requisito 4)

Para a construção da interface, cumprindo o requisito 4, foi utilizado o Bootstrap 5, complementado pelos *plugins Bootstrap Icons* e *Bootstrap Table* (com localização em português). Essas bibliotecas possibilitaram a criação de formulários responsivos, tabelas com paginação e busca no lado do cliente, botões, menus suspensos (*dropdowns*) e alertas de sucesso/erro. Com o auxílio do Bootstrap, foi possível manter um padrão visual consistente entre as telas, agradável e organizado para os usuários.

Figura 4 - Tela de listagem de raças com tabela *Bootstrap Table* paginada



Fonte: Elaborado pelo autor, 2026

2.5 Boas práticas de usabilidade (Requisito 5)

Para atender ao requisito 5, o sistema foi desenvolvido observando boas práticas de usabilidade, como interface limpa e organizada, menus de fácil navegação (agrupados por módulo: Espécies, Raças, Pets e Consultas), padronização visual entre as telas de listagem, criação e edição, mensagens de sucesso e erro para orientação do usuário (via *flashdata* da

sessão) e validação de campos com destaque visual (*class is-invalid*) e mensagens específicas por campo. Também foram adotadas boas práticas de organização de código, como a separação em camadas e a reutilização de trechos de *view* (*layouts* e *partials*), contribuindo para um sistema intuitivo, com redução de erros de uso e de fácil manutenção.

Figura 5 - Exemplo de formulário com validação exibindo mensagem de erro

A imagem mostra uma interface web para criar um novo pet. No topo, há uma barra de navegação com o logo 'FARO ANIMAL' e links para 'Início', 'Espécies', 'Raças', 'Pets' e 'Consultas'. Abaixo, uma mensagem de erro vermelha indica 'Erro!' e pede para 'Verifique os campos'. O formulário 'Novo Pet' contém os seguintes campos:

- Espécie:** Menu suspenso com a mensagem 'Selecione uma espécie'.
- Raça:** Menu suspenso com a mensagem 'Selecione uma espécie primeiro'. Abaixo dele, uma mensagem de erro vermelha diz 'Este campo é obrigatório.'.
- Nome:** Campo de texto com a mensagem 'Digite o nome do pet'. Abaixo dele, uma mensagem de erro vermelha diz 'Este campo é obrigatório.'.
- Peso:** Campo de texto com o valor '0.01 kg'. Abaixo dele, uma mensagem de erro vermelha diz 'Este campo é obrigatório.'.
- Nome do tutor:** Campo de texto com a mensagem 'Digite o nome do tutor'. Abaixo dele, uma mensagem de erro vermelha diz 'Este campo é obrigatório.'.
- Data de nascimento:** Campo de texto com o formato 'dd/mm/aaaa' e um ícone de calendário. Abaixo dele, uma mensagem de erro vermelha diz 'Este campo é obrigatório.'.
- Sexo:** Menu suspenso com a mensagem 'Selecione o sexo'. Abaixo dele, uma mensagem de erro vermelha diz 'Este campo é obrigatório.'.
- Contato do tutor:** Campo de texto com o formato '(00) 00000-0000'. Abaixo dele, uma mensagem de erro vermelha diz 'Número de telefone inválido.'.
- Anotações:** Área de texto com a mensagem 'Faça as anotações...'. Abaixo dela, uma mensagem de erro vermelha diz '0 / 1000 caracteres' e 'A nota precisa ter pelo menos 3 caracteres.'.

Fonte: Elaborado pelo autor, 2026

2.6 Páginas públicas (Requisito 6)

O sistema desenvolvido possui mais páginas públicas do que o mínimo exigido pelo requisito 6, que solicita ao menos duas: a página inicial (rota '/', *HomeController::index*), conforme Figura 1, e a página de *login* (rota '/login', *AuthController::index*), conforme Figura 6, onde o usuário informa e-mail e senha para acessar a área restrita. Além dessas, também estão disponíveis sem autenticação as páginas de cadastro de novo veterinário ('/register'), de recuperação de senha ('/forgot/password') e de definição de nova senha a partir do token recebido por e-mail ('/reset/{token}'), todas fora do grupo de rotas protegido pelo filtro 'auth'.

Figura 6 - Página de *login*

Localhost:8080/index.php/login?

FARO ANIMAL Início Espécies Raças Pets Consultas Cadastre-se Entrar

Faça o login

Acesse as informações da clínica.

E-mail
Digite seu e-mail

Senha
Digite sua senha

Entrar

[Esqueceu a senha?](#)

Início Espécies Raças Pets Consultas

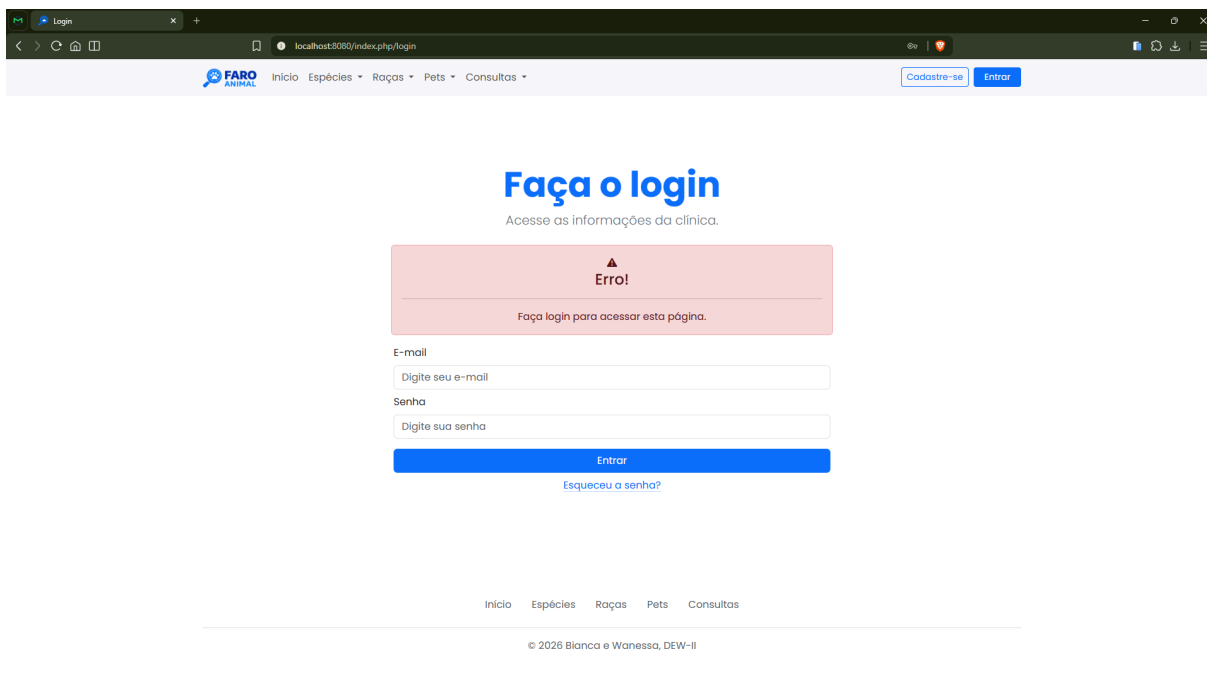
© 2026 Bianca e Wanessa, DEW-II

Fonte: Elaborado pelo autor, 2026

2.7 Área restrita (Requisito 7)

Após a autenticação do usuário, o sistema o direciona para a área restrita administrativa. Essa proteção é implementada por meio do *AuthFilter* (*App\Filters\AuthFilter*), registrado como filtro de rota '*auth*' e aplicado a todo o grupo de rotas protegidas em *Routes.php* (perfil do usuário, espécies, raças, pets e consultas). O filtro verifica a existência da chave '*auth*' na sessão antes de processar a requisição, caso o usuário não esteja autenticado, ele é redirecionado para a tela de login com uma mensagem informativa. Dessa forma, usuários não autenticados não conseguem acessar diretamente nenhuma das páginas de gerenciamento da clínica, mesmo digitando a URL manualmente.

Figura 7 - Tentativa de acesso a rota protegida sem login, sendo redirecionado

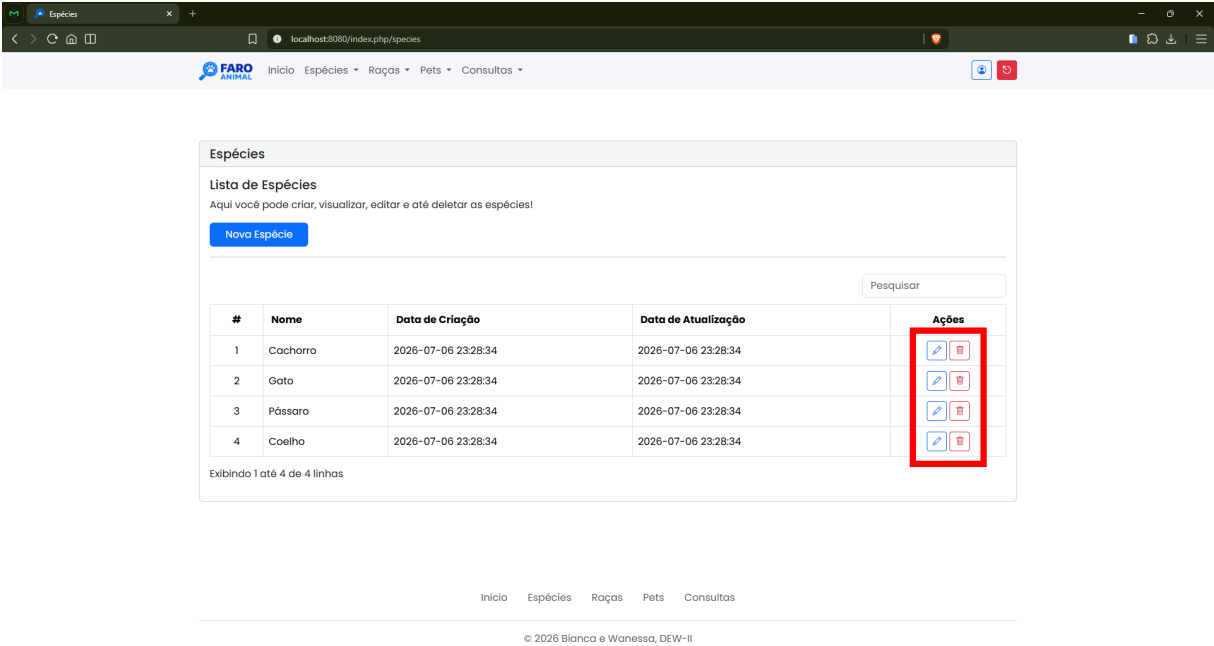


Fonte: Elaborado pelo autor, 2026

2.8 Operações de CRUD (Requisito 8)

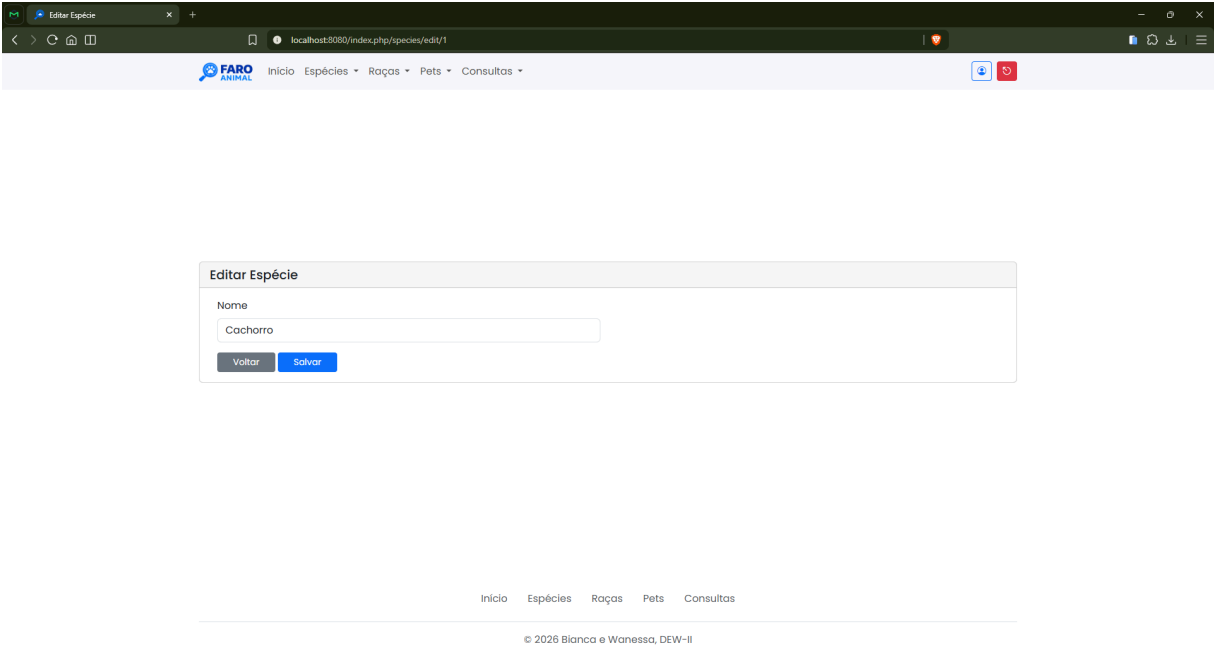
Como definido no requisito 8, foram implementadas quatro operações completas de CRUD (criação, listagem, edição e exclusão): Espécies, Raças, Pets e Consultas, além de um CRUD parcial de gerenciamento do próprio perfil do usuário (edição de nome, alteração de senha e exclusão de conta). Em cada módulo é possível inserir novos registros, consultar os registros já cadastrados, alterar suas informações e realizar a exclusão (implementada como *soft delete*, preservando o histórico no banco através da coluna *deleted_at*). Com essas funcionalidades, é possível o gerenciamento completo das informações clínicas dos pets atendidos.

Figura 8 - Tela de listagem de espécies com as ações de editar e excluir



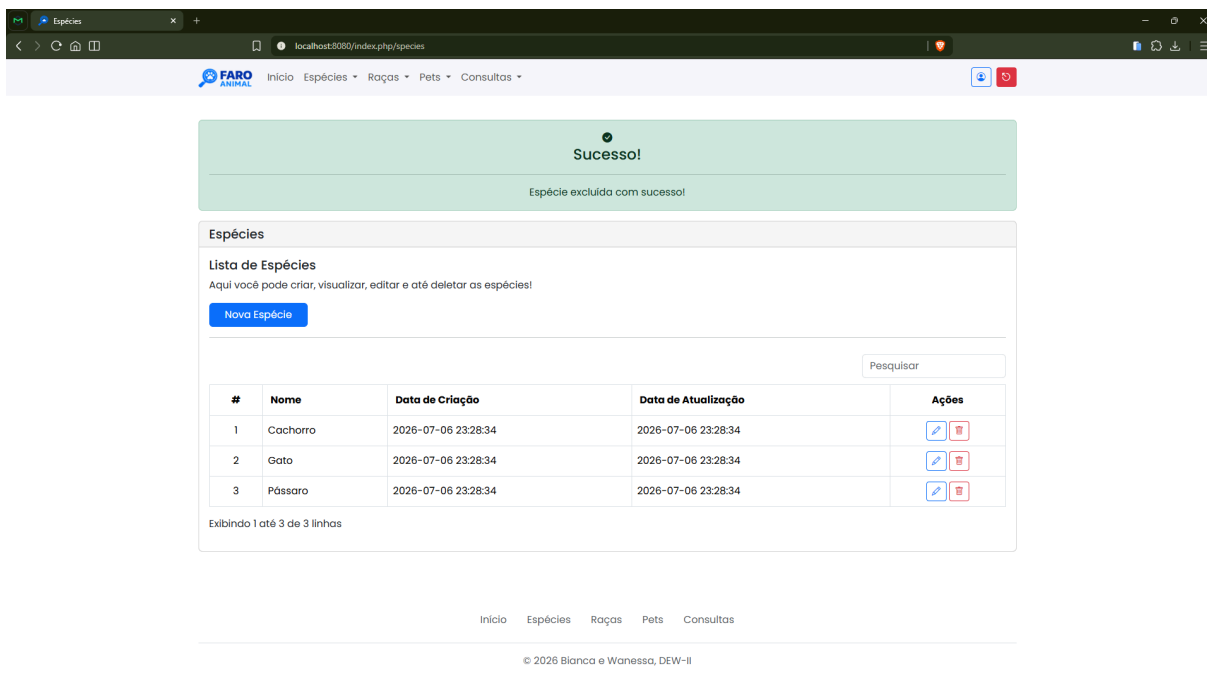
Fonte: Elaborado pelo autor, 2026

Figura 9 - Tela de edição de espécie



Fonte: Elaborado pelo autor, 2026

Figura 10 - Confirmação de exclusão de espécie

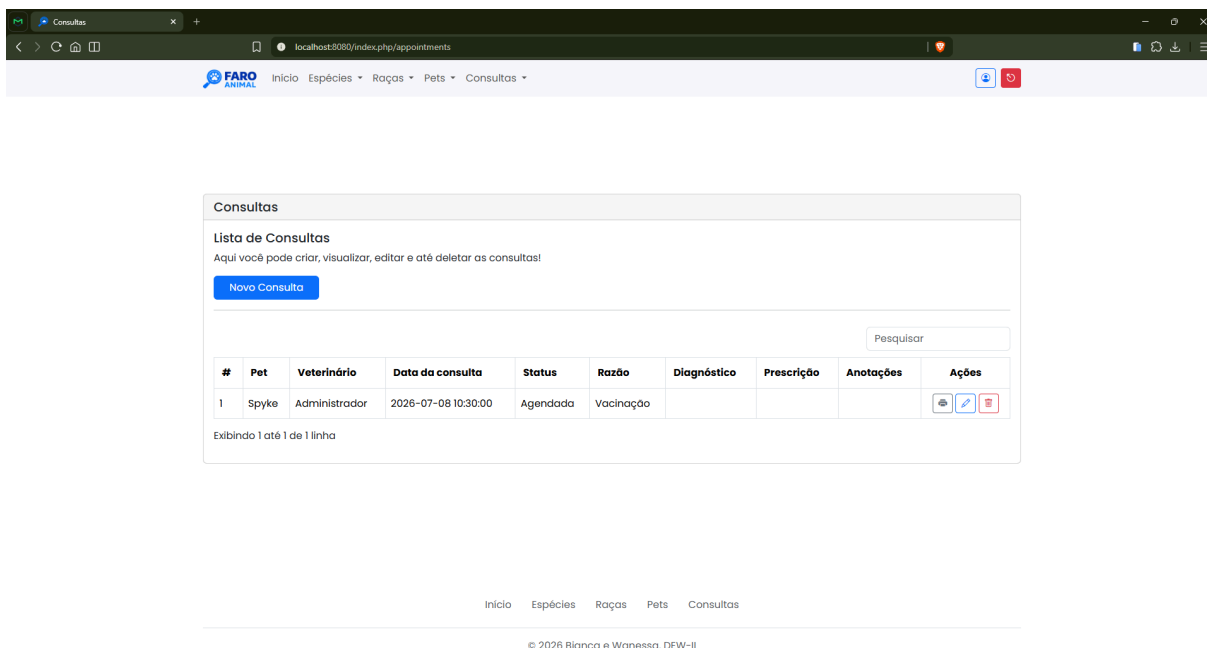


Fonte: Elaborado pelo autor, 2026

2.9 Relacionamento entre tabelas (Requisito 9)

O requisito 9 define que o sistema deve possuir, no mínimo, duas tabelas com relacionamento 1:N. O sistema desenvolvido possui, na verdade, três relacionamentos desse tipo: espécies e raças (uma espécie pode ter várias raças, *breeds.species_id*), raças e pets (uma raça pode ter vários pets, *pets.breed_id*) e pets e consultas (um pet pode ter várias consultas, *appointments.pet_id*), além do relacionamento entre usuários (veterinários) e consultas (*appointments.user_id*). O CRUD de Consultas é o que melhor evidencia esse relacionamento, pois, ao cadastrar ou listar uma consulta, os dados são combinados por meio de *joins* com as tabelas *pets* e *users*, exibindo o nome do *pet* e do veterinário responsável em uma única tela.

Figura 11 - Tela de listagem de consultas



Fonte: Elaborado pelo autor, 2026

2.10 Exportação em PDF (Requisito 10)

Para cumprir o requisito 10, o sistema realiza a exportação da ficha de uma consulta em formato PDF, utilizando a biblioteca *Dompdf*. O método *AppointmentController::export* busca os dados completos da consulta (incluindo *pet*, tutor e veterinário) por meio do *AppointmentService*, renderiza a *view appointments/export* (formatada com CSS específico para impressão) e utiliza o *Dompdf* para converter esse HTML em um arquivo PDF no formato A4, disponibilizado diretamente no navegador do usuário (aberto em nova aba, sem forçar o *download*). Essa funcionalidade facilita a rotina clínica, permitindo a impressão ou o armazenamento da ficha de cada atendimento.

Figura 12 - PDF gerado de uma ficha de consulta

Ficha de Consulta
Documento gerado pelo sistema Faro Animal

Dados da Consulta	
Pet	Spyke
Tutor	Bianca
Veterinário	Administrador
Data/Horário	2026-07-08 10:30:00
Status	Agendada

Informações da Consulta

Razão da consulta

Vacinação

Diagnóstico

Prescrição

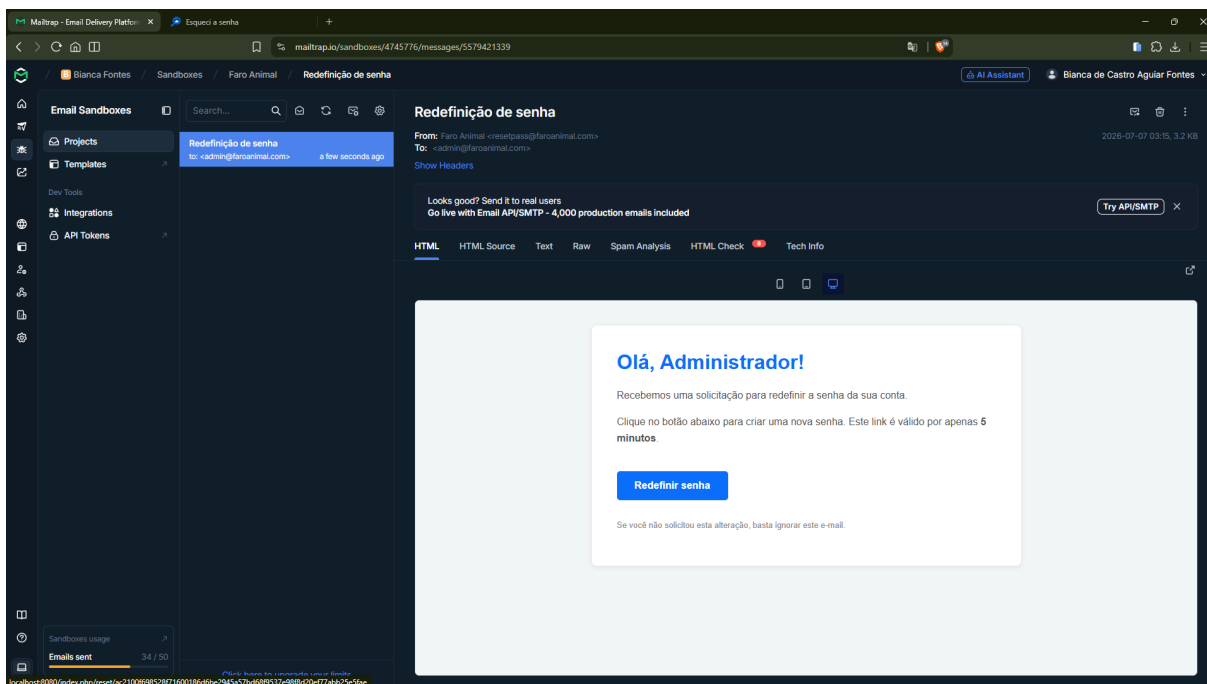
Anotações

Fonte: Elaborado pelo autor, 2026

2.11 Autenticação de usuário (Requisito 11)

O sistema possui controle completo de sessão, atendendo ao requisito 11. O *login* é realizado pelo *AuthService::auth*, que valida e-mail e senha (com senha armazenada com *hash* via *password_hash*, através de um *callback beforeInsert/beforeUpdate* no *UserModel*) e grava os dados do usuário na sessão. O *logout* destrói a sessão ativa. Para a recuperação de senha, o usuário informa seu e-mail cadastrado; o sistema gera um *token* aleatório com validade de 5 minutos (armazenado nas colunas *reset_token* e *reset_expires_at*) e envia, por e-mail, um *link* de redefinição utilizando a biblioteca *Mail* (*App\Libraries\Mail*), que encapsula o serviço de e-mail nativo do CodeIgniter 4 configurado via SMTP com credenciais definidas no arquivo *.env*. Ao acessar o *link*, o *token* é validado (existência e prazo de expiração) antes de permitir a definição de uma nova senha.

Figura 13 - E-mail de redefinição de senha recebido pelo usuário



Fonte: Elaborado pelo autor, 2026

2.12 Requisições AJAX (Requisito 12)

O requisito 12 define que o sistema deve conter comunicação assíncrona por meio de requisições AJAX. Essa funcionalidade foi implementada no cadastro e na edição de *pets*: ao selecionar uma espécie no campo 'Espécie', o formulário dispara uma requisição assíncrona (*fetch*) para a rota *GET /breeds/specie/(:num)*, sem recarregar a página, enviando o cabeçalho *X-Requested-With* para identificar a chamada como AJAX.

No lado do servidor, o método *BreedController::getBySpecie* verifica se a requisição é, de fato, uma chamada AJAX (*\$this->request->isAJAX()*), retornando um erro 403 caso contrário, e delega a busca ao *BreedService::getBreedsBySpecie*, que consulta apenas as raças (*id* e *name*) vinculadas à espécie selecionada. O resultado é devolvido em formato JSON e utilizado pelo *JavaScript* da *view pets/create.php* para popular dinamicamente o campo 'Raça', que permanece desabilitado até que uma espécie seja escolhida. Dessa forma, o usuário só visualiza as raças compatíveis com a espécie selecionada, sem a necessidade de recarregar a página, melhorando a experiência de preenchimento do formulário.

Figura 14 - Campo de raças sendo preenchido dinamicamente após selecionar a espécie

The screenshot shows a web browser window with the URL `localhost:8080/index.php/pets/create`. The page title is 'Novo Pet'. The form contains the following elements:

- Espécie:** A dropdown menu with 'Cachorro' selected.
- Raça:** A dropdown menu that is open, displaying a list of breeds: Beagle, Bulldog Francês, Golden Retriever, Labrador Retriever, Pastor Alemão, Poodle, Shih Tzu, and SRD (Sem Raça Definida).
- Nome:** A text input field with the placeholder 'Digite o nome do pet'.
- Peso:** A text input field with the value '0.01 kg'.
- Nome do tutor:** A text input field with the placeholder 'Digite o nome do tutor'.
- Anotações:** A text area with the placeholder 'Faça as anotações...' and a character count of '0 / 1000 caracteres'.
- Buttons:** 'Voltar' (grey) and 'Cadastrar' (blue).

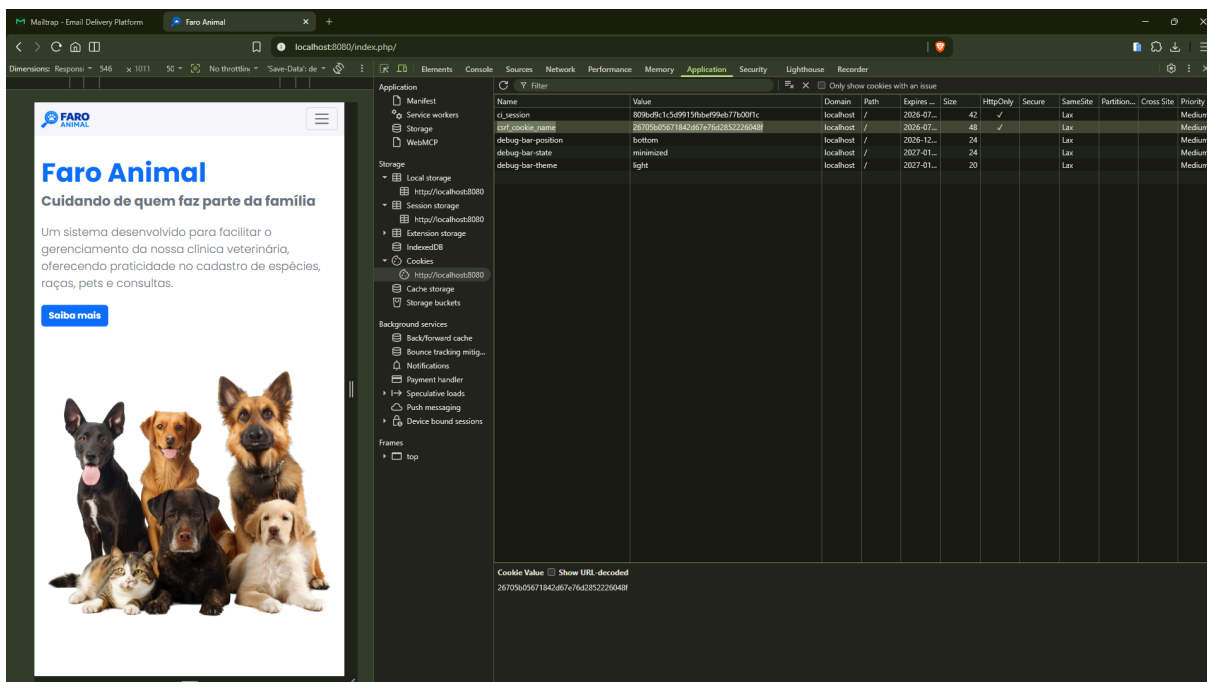
The footer of the page includes the navigation menu: 'Início', 'Espécies', 'Raças', 'Pets', 'Consultas', and the copyright notice: '© 2026 Blanca e Wanessa, DEW-II'.

Fonte: Elaborado pelo autor, 2026

2.13 Filtro de proteção contra CSRF (Requisito 13)

O requisito 13 foi cumprido por meio da proteção de todos os formulários da aplicação contra ataques do tipo CSRF (*Cross-Site Request Forgery*). O CodeIgniter 4 gera um *token* de segurança único por sessão/requisição, inserido em cada formulário através da função `csrf_field()`, e todas as rotas de escrita (POST e DELETE) declaradas em `Routes.php` exigem explicitamente o filtro `'filter' => 'csrf'`. Como os formulários HTML nativos não suportam o verbo DELETE, foi utilizado o recurso de *method spoofing* do CodeIgniter 4 (campo oculto `_method` com valor DELETE) nas ações de exclusão, mantendo a proteção CSRF também nessas operações. Assim, o servidor garante que apenas requisições legítimas, originadas da própria aplicação, sejam processadas.

Figura 15 - Token CSRF gerado ao efetuar login



Fonte: Elaborado pelo autor, 2026

2.14 Proteção contra XSS

Embora não haja um requisito específico do roteiro dedicado a esse ponto, a equipe optou por implementar também, após rápida discussão em sala com o professor, proteção contra ataques do tipo XSS (*Cross-Site Scripting*), já que diversos campos do sistema (como anotações, razão da consulta, diagnóstico e prescrição) aceitam texto livre digitado pelo usuário e são posteriormente exibidos para outros usuários autenticados.

Para isso, foi utilizada a função `esc()`, nativa do CodeIgniter 4, em dois pontos principais da aplicação. No preenchimento dos formulários de criação e edição, todos os valores reaproveitados através da função `old()` são escapados antes de serem inseridos nos atributos `value` dos campos, evitando que um valor malicioso encerre o atributo HTML e injete código executável. Nas telas de listagem (espécies, raças, pets e consultas), que exibem diretamente os dados vindos do banco em formato de tabela, cada campo dinâmico também passou a ser tratado com `esc()` antes de ser impresso, incluindo os campos de texto livre mais sensíveis, como anotações, diagnóstico e prescrição. A mesma prática foi adotada no *template* do *e-mail* de recuperação de senha e na ficha de consulta exportada em PDF, ambos exibindo

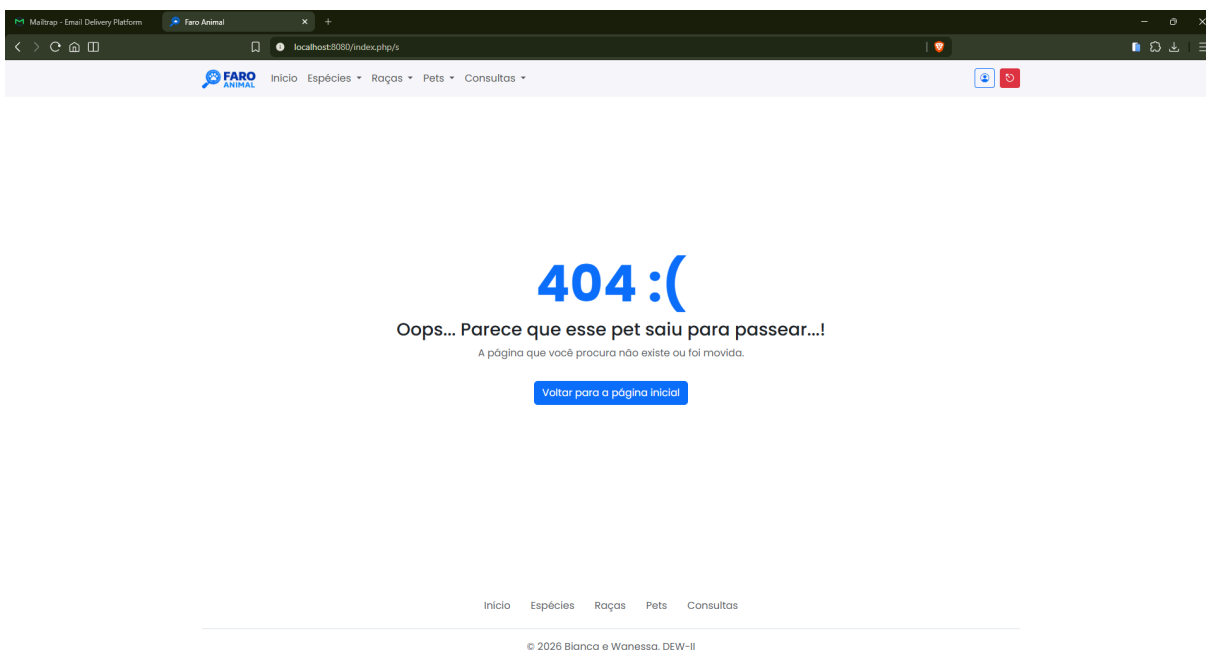
dados fornecidos pelo próprio usuário (nome, motivo da consulta, diagnóstico, prescrição e anotações).

Com essas medidas, o sistema reduz significativamente o risco de um usuário conseguir armazenar um *script* malicioso em um dos campos de texto livre e tê-lo executado no navegador de outro usuário ao visualizar a mesma informação, reforçando a segurança da aplicação para além do que foi exigido pelo requisito 13.

2.15 Página de *error* personalizada (Requisito 14)

Por fim, para cumprir o requisito 14, foi criada uma página personalizada para o *error* '404 - Página não encontrada'. O método *set404Override()*, configurado em *Routes.php*, intercepta qualquer tentativa de acesso a uma rota inexistente, define o código de status HTTP 404 e renderiza a *view errors/custom404*, construída sobre o mesmo *layout* principal do sistema, exibindo uma mensagem temática ('Oops... Parece que esse pet saiu para passear...!') e um botão de retorno para a página inicial, mantendo a identidade visual do sistema mesmo em situações de erro.

Figura 16 - Página 404 personalizada



3 FUNCIONALIDADES NÃO CONCLUÍDAS, LIMITAÇÕES E DIFICULDADES

Conforme solicitado no roteiro do trabalho, esta seção descreve, de forma transparente, os pontos que não foram totalmente concluídos, as limitações identificadas na versão atual do sistema e as principais dificuldades enfrentadas pela equipe durante o desenvolvimento.

3.1 Funcionalidades não concluídas ou com escopo reduzido

- O uso de AJAX no sistema ficou concentrado em um único fluxo (carregamento de raças por espécie); outras listagens, como a de consultas e pets, ainda dependem de recarregamento de página para novas buscas ou filtros no servidor.
- Não foi implementado um painel (*dashboard*) com indicadores gerais da clínica, como quantidade de consultas por período ou por *status*.
- O cadastro de pets não permite o envio de fotos do animal, ficando restrito a dados textuais e numéricos.
- Não há diferenciação de papéis de usuário (por exemplo, veterinário e recepcionista); todo usuário autenticado possui as mesmas permissões dentro da área restrita.
- Não há implementação da recuperação de usuário deletado, o usuário é notificado que o *e-mail* já foi cadastrado, mas não é possível recuperar a conta.
- Não há listagem de consultas por usuário, o usuário consegue filtrar buscando seu nome, porém não existe uma *view* dedicada à listagem de consultas do usuário da sessão.

3.2 Limitações identificadas

- A exclusão dos registros é feita via *soft delete* (coluna *deleted_at*); os dados não são removidos fisicamente do banco, o que exige atenção ao volume de dados em uma eventual utilização de longo prazo.
- O envio de *e-mails* de recuperação de senha depende de configuração externa de SMTP (variáveis de ambiente no *.env*); em ambientes sem esse serviço configurado corretamente, a funcionalidade de recuperação de senha fica indisponível.
- A paginação e a busca nas tabelas de listagem são realizadas no lado do cliente (*Bootstrap Table*), o que pode impactar o desempenho caso o volume de registros cresça significativamente, já que todos os dados são carregados do servidor de uma só vez.
- Não foram implementados testes automatizados (unitários ou de integração) para as classes de *Service* e *Controller*.

3.3 Principais dificuldades enfrentadas

Durante o desenvolvimento, a principal dificuldade enfrentada pela equipe esteve relacionada à implementação do fluxo de autenticação e recuperação de senha via *e-mail*. Os tutoriais encontrados centralizavam a lógica no *controller*, para manter a lógica de negócios no *service* foi desempenhado um tempo significativo.

4 CONCLUSÃO

O desenvolvimento deste projeto possibilitou colocar em prática os conteúdos estudados durante o semestre na disciplina. No decorrer da implementação, foram aplicados conceitos como arquitetura MVC, utilização do *framework* CodeIgniter 4, manipulação de banco de dados relacional, autenticação de usuários, desenvolvimento de interfaces com personalização utilizando o Bootstrap 5, geração de documentos em PDF, comunicação assíncrona via AJAX e proteção contra ataques CSRF e XSS.

O sistema foi desenvolvido atendendo a todos os 14 requisitos definidos no roteiro proposto, oferecendo uma solução para o gerenciamento de uma clínica veterinária que permite o cadastro de espécies, raças, *pets* e consultas de forma organizada, segura e intuitiva. Durante o desenvolvimento, foram encontrados desafios principalmente quanto à autenticação de usuários utilizando *e-mail* e à padronização das regras de negócio nas camadas de *Service*, conforme detalhado na seção anterior. Esses desafios, no entanto, foram essenciais para o aprendizado da equipe, colocando à prova o conteúdo estudado em sala de aula.

Como trabalhos futuros, destacam-se a implementação de um painel de indicadores, a diferenciação de papéis de usuário, a adoção de paginação no lado do servidor, recuperação de conta e a criação de testes automatizados, de modo a tornar o sistema ainda mais robusto e escalável para um uso real em uma clínica veterinária.